

Taller Práctico 4 – MovieLibrary

Arquitectura MVVM con Navigation, Safe Args, LiveData y Room

0. Contexto y objetivos generales

En este taller el estudiante diseñará e implementará **desde cero** una pequeña aplicación Android llamada **MovieLibrary**, completamente independiente de los talleres anteriores.

La aplicación servirá para gestionar un mini catálogo de películas almacenado **en una base de datos local** (Room) y navegar entre distintas pantallas usando **Navigation Component** y **Safe Args**.

El foco del taller no está en el diseño visual sino en la **arquitectura** y en el uso correcto de:

- **Bundle** y **Safe Args** para pasar datos entre fragments.
 - **NavController** como motor de navegación.
 - Uso explícito y necesario de **LiveData**.
 - Patrón **Repository** apoyado en una base de datos local **Room**.
-

1. Idea de la aplicación MovieLibrary

La aplicación **MovieLibrary** permite llevar un registro sencillo de películas que la persona ha visto o quiere ver.

Cada película tendrá, como mínimo, los siguientes campos:

- **id** (entero, identificador único, generado por la base de datos).
- **title** (texto, título de la película).
- **year** (entero, año de estreno aproximado).
- **genre** (texto, género principal, por ejemplo: Acción, Drama, Comedia).
- **rating** (entero o flotante, calificación del 1 al 5 o del 1 al 10).
- **watched** (booleano, indica si la película ya fue vista o no).

La aplicación **no** se conectará a internet ni a APIs externas: toda la información se almacenará y leerá **desde una base de datos local Room**.

2. Vistas obligatorias de la app

La aplicación deberá tener, como mínimo, las siguientes pantallas (Fragments):

1. **MovieListFragment** (lista de películas)

- Muestra la lista completa de películas almacenadas en la base de datos (RecyclerView).
 - Permite seleccionar una película para ver su detalle.
 - Incluye un botón flotante o similar para “Agregar película”.
2. **MovieDetailFragment (detalle de película)**
 - Muestra la información completa de una película seleccionada.
 - Debe permitir:
 - Marcar o desmarcar la película como vista (**watched**).
 - Navegar a una pantalla de edición para modificar los datos.
 - Opcional: eliminar la película.
 3. **MovieEditFragment (crear / editar película)**
 - Muestra un formulario con campos para:
 - Título.
 - Año.
 - Género.
 - Rating.
 - Indicador de si está vista o no.
 - En modo “crear”: el formulario está vacío y al guardar se inserta una nueva película.
 - En modo “editar”: el formulario se llena con los datos de una película existente y al guardar se actualiza.

Además, la aplicación contará con una única **MainActivity** que hospedará un **NavHostFragment** configurado con el **Navigation Graph** del proyecto.

3. Requisitos de arquitectura y tecnologías

La implementación del taller debe cumplir con los siguientes lineamientos:

1. **Arquitectura MVVM**
 - Separar claramente:
 - **model** (datos, entidades de dominio).
 - **repository** (acceso a datos, Room).
 - **viewmodel** (lógica de presentación y estado de UI).
 - **ui** (Fragments y Activities).
2. **Room como base de datos local**
 - Definir al menos una **Entity** para las películas (**MovieEntity**).
 - Definir un **Dao** con las operaciones básicas (consultar, insertar, actualizar, eliminar).
 - Definir una clase **AppDatabase** que extienda de **RoomDatabase** y exponga el **MovieDao**.
3. **Repository**
 - Implementar una clase **MovieRepository** que reciba el **MovieDao** y oculte los detalles de acceso a datos.
 - El **ViewModel** no debe acceder a Room directamente.

4. **ViewModel + LiveData**
 - Usar **ViewModel** para mantener el estado de la lista de películas y de la película seleccionada.
 - Exponer la información a la UI por medio de **LiveData**, de forma que las vistas se actualicen automáticamente cuando cambien los datos.
 5. **Navigation Component + NavController**
 - Definir un **nav_graph.xml** con los tres fragments requeridos.
 - Establecer **MovieListFragment** como **startDestination**.
 - Utilizar **NavController** para ejecutar la navegación entre fragments.
 6. **Bundle y Safe Args**
 - Primero, implementar el paso de argumentos utilizando un **Bundle** manual (para comprender los riesgos).
 - Luego, migrar a **Safe Args**, definiendo los argumentos en el Navigation Graph y usando las clases generadas.
 - Los argumentos obligatorios mínimos:
 - De lista a detalle: **movieId** (Int).
 - De detalle a edición: **movieId** (Int) o, si se prefiere, un objeto parcelable.
-

4. Casos de uso que la app debe soportar

La aplicación deberá satisfacer, al menos, los siguientes casos de uso:

1. **Ver la lista inicial de películas**
 - Al abrir la app, se muestra **MovieListFragment** con la lista de películas.
 - Si aún no hay películas, mostrar un mensaje adecuado (por ejemplo, “No hay películas registradas”).
2. **Agregar una nueva película**
 - Desde **MovieListFragment**, pulsar el botón “Agregar película”.
 - Navegar a **MovieEditFragment** en modo creación.
 - Completar el formulario y guardar.
 - Volver a la lista y mostrar la nueva película, confirmando que los datos se guardaron en la base de datos local.
3. **Ver detalle de una película**
 - Desde la lista, seleccionar una película específica.
 - Navegar a **MovieDetailFragment** usando **NavController**.
 - Pasar el identificador de la película (**movieId**) mediante **Bundle** primero, y posteriormente mediante **Safe Args**.
 - Mostrar todos los datos de la película.
4. **Marcar una película como vista / no vista**
 - En **MovieDetailFragment**, permitir cambiar el estado **watched**.
 - Guardar el cambio en la base de datos (vía **ViewModel + Repository + Room**).

- Al regresar a la lista, reflejar visualmente el estado (por ejemplo, con un icono o un texto).
5. **Editar una película existente**
 - Desde el detalle, pulsar un botón “Editar”.
 - Navegar a `MovieEditFragment` con los datos de la película cargados.
 - Modificar algunos campos y guardar.
 - Regresar al detalle o a la lista y verificar que los cambios fueron persistidos.
 6. **(Opcional) Eliminar una película**
 - Desde el detalle, un botón “Eliminar” que suprima la película de la base de datos y regrese a la lista.
-

5. Partes del taller y pasos sugeridos

A continuación se propone una estructura de trabajo en partes. No se incluyen soluciones completas; se describen los resultados esperados en cada sección.

Parte 1 – Configuración base del proyecto

1. Crear un nuevo proyecto Android con Activity vacía (`Empty Activity`) y nombre de paquete coherente.
2. Agregar las dependencias necesarias en `build.gradle` (módulo `app`):
 - Room (`room-runtime`, `room-ktx`, `kapt` para el compiler si se usa).
 - Lifecycle (`viewmodel-ktx`, `livedata-ktx`).
 - Navigation Component (`navigation-fragment-ktx`, `navigation-ui-ktx`).
3. Habilitar `viewBinding` o `dataBinding` según se prefiera.
4. Crear los paquetes:
 - `model`
 - `data` o `repository`
 - `db` (opcional, para Room)
 - `viewmodel`
 - `ui` (subpaquetes para los fragments)

Parte 2 – Modelo de datos y Room

1. En `model`, crear la clase de dominio `Movie` (data class simple, sin anotaciones de Room).
2. En `db` o `data`, crear `MovieEntity` con anotaciones `@Entity` y las columnas necesarias.
3. Crear `MovieDao` con los métodos mínimos:
 - `getAllMovies(): LiveData<List<MovieEntity>>`
 - `getMovieById(id: Int): LiveData<MovieEntity>`
 - `insert(movie: MovieEntity)`
 - `update(movie: MovieEntity)`
 - `delete(movie: MovieEntity)`

4. Crear `AppDatabase` que extienda `RoomDatabase` y provea una instancia de `MovieDao`.
5. Implementar funciones de conversión `MovieEntity` `Movie`.

Parte 3 – Repository y ViewModel

1. Crear `MovieRepository` que reciba un `MovieDao` (por constructor o por inyección simple) y exponga:
 - `fun getMovies(): LiveData<List<Movie>>`
 - `fun getMovie(id: Int): LiveData<Movie>`
 - `suspend fun insert(movie: Movie)`
 - `suspend fun update(movie: Movie)`
 - `suspend fun delete(movie: Movie)`
2. Crear `MovieViewModel` que reciba un `MovieRepository` y defina:
 - `val movies: LiveData<List<Movie>> = repository.getMovies()`
 - Métodos públicos para cargar una película concreta y para crear/editar/eliminar.
3. Asegurarse de que todas las operaciones de escritura se hagan usando `viewModelScope.launch { ... }` para no bloquear el hilo principal.

Parte 4 – Fragments y layouts

1. Crear `MovieListFragment`, `MovieDetailFragment` y `MovieEditFragment` dentro del paquete `ui`.
2. Diseñar layouts XML sencillos para cada fragmento:
 - Lista: `RecyclerView` + indicador de conteo + botón de agregar.
 - Detalle: textos con la información de la película + botones de acción.
 - Edición: formulario con campos de entrada.
3. Conectar `MovieListFragment` con el `MovieViewModel` y observar `movies` para poblar el `RecyclerView`.

Parte 5 – Navigation, Bundle y Safe Args

1. Crear el archivo de Navigation Graph (`nav_graph.xml`) en `res/navigation`.
2. Registrar en el graph los tres fragments e indicar a `MovieListFragment` como `startDestination`.
3. Configurar `MainActivity` para usar un `NavHostFragment` con dicho graph.
4. Implementar la navegación **usando Bundle manual**:
 - De lista a detalle pasando `movieId` en un `Bundle`.
 - De detalle a edición, también con `Bundle`.
5. Probar que la navegación funcione y que `MovieDetailFragment` y `MovieEditFragment` lean el identificador mediante `arguments`.
6. Configurar el plugin de **Safe Args** en el proyecto.
7. Definir los argumentos correspondientes en el `nav_graph.xml`.
8. Reemplazar el código de **Bundle manual** por el uso de las clases generadas por **Safe Args**.

Parte 6 – Integración, pruebas y refinamiento

1. Verificar el flujo completo:
 - Agregar película → ver en lista.
 - Seleccionar película → ver detalle.
 - Marcar como vista → comprobar cambio en detalle y lista.
 - Editar película → comprobar actualización persistida.
 2. Probar rotación de pantalla y cierre/reapertura de la app para confirmar que los datos persisten gracias a Room y que la UI se actualiza con LiveData.
 3. Revisar que ningún Fragment acceda directamente a Room y que todo pase por el Repository y el ViewModel.
-

6. Entregables sugeridos

Se recomienda que el taller tenga como entregables mínimos:

1. **Repository GitHub actualizado**
 - Código fuente del proyecto MovieLibrary.
 - Estructura de paquetes clara y coherente con MVVM.
 2. **Video corto de demostración (2–3 minutos)**
 - Recorrido por la aplicación mostrando los casos de uso requeridos.
 - Evidencia de que los datos persisten y que la UI se actualiza automáticamente.
 3. **Documento de reflexión breve**
 - Explicación, en lenguaje propio, del rol de:
 - ViewModel.
 - LiveData.
 - Repository.
 - Navigation Component, NavController, Safe Args.
 - Room (Entity, DAO, Database).
-